

B Track : AI Agent 아키텍트 과정 — 학습 노트

Notion 원본: <https://teamparta.notion.site/B-Track-AI-Agent-3c92dc3ef514818fa341c9203d4faa92>

교안 링크: <https://b2bsparta.com/0827-686a>

삼성 DS 대상 1일(8교시) 바이브코딩 교육. Claude Code로 코드를 직접 작성하지 않고
웹앱 → RAG → Agent → 캡스톤까지 4개 프로젝트를 순서대로 만든다.

두 가지 AI, 역할이 다르다

- **Claude Code** — 개발 도구. 자연어 지시로 코드를 작성/수정. 구독 계정 로그인.
- **OpenAI GPT (`gpt-5.6-luna`)** — 앱에 탑재되는 AI 기능. 채팅 응답·임베딩·도구 선택. 배부된 API Key 사용.

이 구분은 8교시까지 계속 유지된다.

시간표 & 노트 목차

교시	시간	주제	노트
1	09:00–09:50	바이브코딩 입문 (웹앱 + OpenAI 연동)	01-바이브코딩-웹앱.md
2	10:00–10:50	RAG 1 — 저장과 검색 5단계	02-RAG1-저장과검색.md
3	11:00–11:50	RAG 2 — 검색 실패 진단과 개선	03-RAG2-검색개선.md
—	12:00–13:00	점심	
4	13:00–13:50	Tool Calling	04-ToolCalling.md
5	14:00–14:50	MCP	05-MCP.md
6	15:00–15:50	Multi-Agent (Deep Research)	06-MultiAgent.md
7	16:00–16:50	캡스톤 1 — 기획과 제작	07-캡스톤1-기획제작.md
8	17:00–18:00	캡스톤 2 — 완성과 제출	08-캡스톤2-완성제출.md

전체 흐름 한눈에 보기

1교시	웹앱 하나(지도+날씨) 만들고 OpenAI 채팅 붙이기	→ API, .env, 바이브코딩 기본기
2교시	rag-lab 앱 분석 → naive RAG 5단계 직접 관찰	→ 파싱·칭킹·임베딩·검색·생성
3교시	검색 실패 진단 → Reranking · Metadata Filter · Query Rewrite 처방, LLM-as-a-Judge로 채점	→ "고치는 순서"가 핵심 기술
4교시	agent-lab 앱 → 도구 설명문이 곧 AI의 판단 근거	→ Tool Calling 구조
5교시	MCP 서버 연결(로컬/원격) → 도구 목록이 그대로 합쳐짐	→ 표준 규격의 힘
6교시	Multi-Agent(연구책임자·자료조사·실험수행) 역할 분담	→ 왜 나누는가(맥락 한계, 유지보수)
7~8교시	캡스톤 - 트랙 선택, 100분 제작, report.html 제출	→ 자유/도구설계/함정질문/조사리포트

자습·실습 가이드 (혼자 따라 하기)

강의 노트가 개념 설명이라면, 아래는 **체크리스트 형식의 실행 가이드**입니다. 배부 계정/배부 파일 없이 본인 환경에서 처음부터 끝까지 진행할 수 있게 재구성했습니다.

- [실습가이드-00-준비.md](#) — 환경 설정, 원본과 달라지는 점
- [실습가이드-01-웹앱.md](#)
- [실습가이드-02-RAG1.md](#)
- [실습가이드-03-RAG2.md](#)
- [실습가이드-04-ToolCalling.md](#)
- [실습가이드-05-MCP.md](#)
- [실습가이드-06-MultiAgent.md](#)
- [실습가이드-07-캡스톤.md](#)

반복해서 등장하는 핵심 원칙 (전 교시 공통)

1. **.env** 는 Git 유출 방지용이지, Claude Code(Agent)로부터 Key를 숨기는 장치가 아니다.
Agent가 파일·셀에 접근 가능하면 Key를 읽을 수 있다.
2. **LLM은 도구 "선택"만 한다. 실행은 앱(코드)이 한다.** AI는 도구의 설명문(description)만 보고 판단하며 코드는 못 본다 → 설명문 품질이 곧 정확도.
3. **행동 범위는 두 겹으로 막는다:** ① 역할 프롬프트(문구, 뚫릴 수 있음) ② 아예 도구를 안 주여주기(확실히, MCP 연결/해제·Multi-Agent 역할분리로 구현).
4. **RAG 답변 품질 = 검색 품질.** 문서가 수천 쪽이어도 AI가 실제로 읽는 건 검색이 골라낸 몇 조각뿐.
5. **"고치는 순서"는 시스템 불문 공통 기술:** 증상 확인 → 원인(생성 vs 검색?) 분리 → 맞는 처방 선택 → 개선 여부를 반드시 숫자로 재확인(라운드 0 기준선 필수, 한 번에 한 가지만 변경).
6. **Multi-Agent로 나누는 이유는 "여러 개가 좋아서"가 아니라 역할마다 필요한 자료·도구·권한이 달라서.** 나눌 이유 없으면 한 Agent가 낫다(통신 비용 발생).